

LAPORAN PRAKTIKUM STRUKTUR DATA
“BFS & DFS”



Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 8 JUNI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum mengenai BFS (Breadth First Search) dan DFS (Depth First Search) ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pemahaman penulis terhadap materi praktikum Struktur Data dan Algoritma, khususnya mengenai teknik penelusuran (traversal) pada struktur data graf menggunakan algoritma BFS dan DFS dalam bahasa pemrograman Java.

Dalam praktikum ini, penulis mempelajari konsep dasar graf serta cara kerja algoritma BFS dan DFS dalam menelusuri setiap simpul yang terdapat pada graf. Selain itu, penulis juga memahami perbedaan mekanisme kedua algoritma tersebut, dimana BFS menggunakan prinsip antrian (queue) untuk menelusuri simpul secara melebar, sedangkan DFS menggunakan prinsip tumpukan (stack) atau rekursi untuk menelusuri simpul secara mendalam. Melalui implementasi program yang dibuat, penulis dapat memahami proses traversal graf secara lebih jelas dan sistematis.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang. Semoga laporan praktikum ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai algoritma BFS dan DFS serta penerapannya dalam pemrograman Java.

Padang, 2026

Tim Penyusun

Naufal Arkan Octyandi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 LATAR BELAKANG	1
1.2 TUJUAN PRAKTIKUM.....	1
1.3 MANFAAT PRAKTIKUM.....	1
BAB II PEMBAHASAN PRAKTIKUM	2
2.1 IMPLEMENTASI ALGORITMA INSERTION SORT	2
2.2 IMPLEMENTASI ALGORITMA SELECTION SORT.....	4
2.3 IMPLEMENTASI ALGORITMA BUBBLE SORT	6
2.4 IMPLEMENTASI GUI PADA ALGORITMA INSERTION SORT	8
BAB III KESIMPULAN.....	10
3.1 KESIMPULAN	10
DAFTAR PUSTAKA.....	11

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Graf merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek dalam bentuk simpul dan sisi. Struktur data ini banyak diterapkan dalam berbagai bidang, seperti jaringan komputer, sistem navigasi, media sosial, dan pemetaan jalur. Untuk mengakses serta memproses data yang terdapat pada graf, diperlukan suatu metode penelusuran yang efektif agar setiap simpul dapat dikunjungi sesuai kebutuhan.

Breadth First Search (BFS) dan Depth First Search (DFS) merupakan dua algoritma dasar yang digunakan untuk melakukan penelusuran pada graf. BFS bekerja dengan mengunjungi simpul berdasarkan tingkat kedekatannya menggunakan struktur data queue, sedangkan DFS bekerja dengan menelusuri simpul secara mendalam terlebih dahulu menggunakan stack atau rekursi. Kedua algoritma tersebut memiliki karakteristik dan cara kerja yang berbeda dalam menjelajahi suatu graf.

Dalam praktikum ini, penulis mempelajari implementasi algoritma BFS dan DFS menggunakan bahasa pemrograman Java serta mengamati proses traversal graf untuk memahami perbedaan cara kerja kedua algoritma tersebut secara lebih jelas.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data graf.
2. Memahami cara kerja algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
3. Melatih kemampuan logika pemrograman dalam implementasi traversal graf menggunakan Java.

1.3 MANFAAT PRAKTIKUM

1. Menambah pemahaman mengenai proses penelusuran pada struktur data graf.
2. Melatih kemampuan dalam membuat program BFS dan DFS menggunakan Java.
3. Membantu memahami perbedaan mekanisme traversal antara BFS dan DFS.
4. Meningkatkan kemampuan dalam menerapkan konsep struktur data dan algoritma pada pemecahan masalah.

BAB II

PEMBAHASAN PRAKTIKUM

2.1 Pembuatan Class Node pada Binary Tree

Pertama-tama dibuat Class Node yang digunakan untuk merepresentasikan sebuah node pada struktur data Binary Tree. Setiap node memiliki tiga atribut utama, yaitu data untuk menyimpan nilai, left untuk menunjuk ke child kiri, dan right untuk menunjuk ke child kanan. Constructor pada class ini digunakan untuk menginisialisasi nilai data pada node serta mengatur child kiri dan kanan menjadi null.

Method setLeft dan setRight berfungsi untuk menambahkan child kiri dan child kanan ke suatu node. Method getLeft, getRight, dan getData digunakan untuk mengambil nilai child kiri, child kanan, dan data yang tersimpan pada node. Selain itu, terdapat method setData yang digunakan untuk mengubah nilai data pada node.

Class ini juga menyediakan beberapa method traversal tree, yaitu printPreorder, printInorder, dan printPostorder. Method printPreorder melakukan traversal dengan urutan Root, Left, Right. Method printInorder melakukan traversal dengan urutan Left, Root, Right. Sedangkan method printPostorder melakukan traversal dengan urutan Left, Right, Root. Ketiga method tersebut menggunakan teknik rekursif untuk mengunjungi seluruh node pada tree dan menampilkan nilainya ke layar.

Selain traversal, terdapat method print yang digunakan untuk menampilkan struktur Binary Tree dalam bentuk diagram teks. Method ini memanfaatkan parameter tertentu untuk mengatur posisi serta bentuk cabang yang ditampilkan. Dengan adanya method ini, hubungan antara parent node dan child node dapat terlihat lebih jelas sehingga memudahkan visualisasi struktur Binary Tree yang telah dibuat.

```

1 package pekan9_2511533005;
2
3 public class Node_2511533005 {
4     int data_3005;
5     Node_2511533005 left_3005;
6     Node_2511533005 right_3005;
7
8     public Node_2511533005(int data_3005) {
9         this.data_3005 = data_3005;
10        left_3005 = null;
11        right_3005 = null;
12    }
13
14    public void setLeft(Node_2511533005 node_3005) {
15        if (left_3005 == null)
16            left_3005 = node_3005;
17    }
18
19    public void setRight(Node_2511533005 node_3005) {
20        if (right_3005 == null)
21            right_3005 = node_3005;
22    }
23
24    public Node_2511533005 getLeft() {
25        return left_3005;
26    }
27
28    public Node_2511533005 getRight() {
29        return right_3005;
30    }
31
32    public int getData() {
33        return data_3005;
34    }
35
36    public void setData(int data) {
37        this.data_3005 = data;
38    }
39
40    void printPreorder(Node_2511533005 node_3005) {
41
42        void printPreorder(Node_2511533005 node_3005) {
43            if (node_3005 == null)
44                return;
45            System.out.print(node_3005.data_3005 + " ");
46            printPreorder(node_3005.left_3005);
47            printPreorder(node_3005.right_3005);
48        }
49
50        void printPostorder(Node_2511533005 node_3005) {
51            if (node_3005 == null)
52                return;
53            printPostorder(node_3005.left_3005);
54            printPostorder(node_3005.right_3005);
55            System.out.print(node_3005.data_3005 + " ");
56        }
57
58        void printInorder(Node_2511533005 node_3005) {
59            if (node_3005 == null)
60                return;
61            printInorder(node_3005.left_3005);
62            System.out.print(node_3005.data_3005 + " ");
63            printInorder(node_3005.right_3005);
64        }
65
66        public String print() {
67            return this.print("", true, "");
68        }
69
70        public String print(String prefix_3005, boolean isTail_3005, String sb_3005) {
71            if (right_3005 != null) {
72                right_3005.print(prefix_3005 + (isTail_3005 ? "|" : " " : " " ), false, sb_3005);
73            }
74            System.out.println(prefix_3005 + (isTail_3005 ? "\\-- " : "/-- ") + data_3005);
75            if (left_3005 != null) {
76                left_3005.print(prefix_3005 + (isTail_3005 ? " " : "|" ), true, sb_3005);
77            }
78            return sb_3005;
79        }
80    }
81 }

```

2.2 Implementasi Algoritma BFS dan DFS pada Graf

Selanjutnya kita membuat Class QuickSort, Class QuickSort digunakan untuk mengurutkan data menggunakan metode Quick Sort. Method swap berfungsi Selanjutnya dibuat Class GraphTraversal yang digunakan untuk merepresentasikan graf serta mengimplementasikan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Graf disimpan menggunakan struktur data Map yang berisi pasangan node dan daftar tetangganya (adjacency list). Struktur ini memungkinkan setiap node dapat terhubung dengan beberapa node lain secara efisien.

Method addEdge berfungsi untuk menambahkan hubungan antar node pada graf tak berarah. Sebelum menambahkan hubungan, method ini memastikan bahwa kedua node telah tersedia di dalam graf. Setelah itu, masing-masing node ditambahkan ke daftar tetangga node lainnya sehingga hubungan dapat diakses dari kedua arah.

Method printGraph digunakan untuk menampilkan struktur graf dalam bentuk adjacency list. Setiap node akan ditampilkan beserta node-node yang terhubung dengannya sehingga hubungan antar node dapat terlihat dengan jelas sebelum proses penelusuran dilakukan.

Method dfs digunakan untuk melakukan penelusuran graf menggunakan algoritma Depth First Search (DFS). Method ini memanfaatkan HashSet untuk menyimpan node yang telah dikunjungi agar tidak terjadi kunjungan berulang. Proses penelusuran dilakukan secara rekursif melalui method dfsHelper, dimana node akan dikunjungi terlebih dahulu kemudian dilanjutkan ke seluruh tetangganya hingga mencapai node terdalam sebelum kembali ke node sebelumnya.

Method bfs digunakan untuk melakukan penelusuran graf menggunakan algoritma Breadth First Search (BFS). Pada algoritma ini digunakan struktur data Queue untuk menyimpan node yang akan dikunjungi. Penelusuran dilakukan secara bertahap berdasarkan tingkat kedekatan node, dimana seluruh tetangga dari suatu node akan dikunjungi terlebih dahulu sebelum berpindah ke tingkat berikutnya.

Pada method main dibuat sebuah objek graf yang berisi beberapa hubungan antar node, yaitu A-B, A-C, B-D, dan B-E. Setelah graf terbentuk, method printGraph dipanggil untuk menampilkan struktur graf awal. Selanjutnya dilakukan proses penelusuran menggunakan algoritma DFS dan BFS yang dimulai dari node A sehingga urutan kunjungan node pada kedua algoritma dapat diamati dan dibandingkan.

```

1 package pekan9_2511533005;
2 import java.util.ArrayList;
3 import java.util.HashMap;
4 import java.util.HashSet;
5 import java.util.LinkedList;
6 import java.util.List;
7 import java.util.Map;
8 import java.util.Queue;
9 import java.util.Set;
10
11 public class GraphTraversal_2511533005 {
12     private Map<String, List<String>> graph_3005 = new HashMap<>();
13
14     // Menambahkan edge (graf tak berarah)
15     public void addEdge(String node1_3005, String node2_3005) {
16         graph_3005.putIfAbsent(node1_3005, new ArrayList<>());
17         graph_3005.putIfAbsent(node2_3005, new ArrayList<>());
18         graph_3005.get(node1_3005).add(node2_3005);
19         graph_3005.get(node2_3005).add(node1_3005);
20     }
21
22     // Menampilkan graf awal
23     public void printGraph() {
24         System.out.println("Graf Awal (Adjacency List):");
25         for (String node_3005 : graph_3005.keySet()) {
26             System.out.print(node_3005 + " -> ");
27             List<String> neighbors_3005 = graph_3005.get(node_3005);
28             System.out.println(String.join(", ", neighbors_3005));
29         }
30         System.out.println();
31     }
32
33     // DFS rekursif
34     public void dfs(String start_3005) {
35         Set<String> visited_3005 = new HashSet<>();
36         System.out.println("Penelusuran DFS:");
37         dfsHelper(start_3005, visited_3005);
38         System.out.println();
39     }
40

```

```

41     private void dfsHelper(String current_3005, Set<String> visited_3005) {
42         if (visited_3005.contains(current_3005)) return;
43         visited_3005.add(current_3005);
44         System.out.print(current_3005 + " ");
45         for (String neighbor_3005 : graph_3005.getOrDefault(current_3005, new ArrayList<>())) {
46             dfsHelper(neighbor_3005, visited_3005);
47         }
48     }
49
50     // BFS iteratif
51     public void bfs(String start_3005) {
52         Set<String> visited_3005 = new HashSet<>();
53         Queue<String> queue_3005 = new LinkedList<>();
54         queue_3005.add(start_3005);
55         visited_3005.add(start_3005);
56         System.out.println("Penelusuran BFS:");
57         while (!queue_3005.isEmpty()) {
58             String current_3005 = queue_3005.poll();
59             System.out.print(current_3005 + " ");
60             for (String neighbor_3005 : graph_3005.getOrDefault(current_3005, new ArrayList<>())) {
61                 if (!visited_3005.contains(neighbor_3005)) {
62                     queue_3005.add(neighbor_3005);
63                     visited_3005.add(neighbor_3005);
64                 }
65             }
66         }
67         System.out.println();
68     }
69
70     // Main
71     public static void main(String[] args) {
72         GraphTraversal_2511533005 graph_3005 = new GraphTraversal_2511533005();
73
74         // Contoh graf: A-B, A-C, B-D, B-E
75         graph_3005.addEdge("A", "B");
76         graph_3005.addEdge("A", "C");
77         graph_3005.addEdge("B", "D");
78         graph_3005.addEdge("B", "E");
79
80         // Cetak graf awal
81         graph_3005.printGraph();
82

```



```

78         // Cetak graf awal
79         System.out.println("Graf Awal adalah: ");
80         graph_3005.printGraph();
81         // Lakukan penelusuran
82         graph_3005.dfs("A");
83         graph_3005.bfs("A");
84     }
85
86 }

```

2.3 Pembuatan dan Pengelolaan Binary Tree

Selanjutnya dibuat Class BTree yang digunakan untuk mengelola struktur data Binary Tree. Class ini memiliki atribut root yang berfungsi sebagai akar (root) dari tree serta atribut currentNode yang digunakan untuk menyimpan node yang sedang aktif atau sedang diproses.

Method search digunakan untuk mencari data tertentu di dalam Binary Tree. Proses pencarian dilakukan secara rekursif melalui method search(Node node, int data). Method ini akan membandingkan nilai data pada node saat ini dengan data yang dicari. Jika data ditemukan maka method akan mengembalikan nilai true. Jika belum ditemukan, pencarian akan dilanjutkan ke child kiri dan child kanan hingga seluruh node telah diperiksa.

Method printInorder, printPreOrder, dan printPostOrder digunakan untuk menampilkan isi Binary Tree berdasarkan metode traversal yang berbeda. Method printInorder menampilkan node dengan urutan Left, Root, Right. Method printPreOrder menampilkan node dengan urutan Root, Left, Right. Sedangkan method printPostOrder menampilkan node dengan urutan Left, Right, Root.

Method getRoot digunakan untuk mengambil node akar dari Binary Tree, sedangkan method setRoot digunakan untuk menentukan node yang akan dijadikan sebagai akar tree. Selain itu, terdapat method getCurrent dan setCurrent yang digunakan untuk mengambil dan mengubah node yang sedang aktif.

Method isEmpty digunakan untuk memeriksa apakah Binary Tree masih kosong atau belum memiliki root. Jika root bernilai null maka method akan mengembalikan nilai true.

Method countNodes digunakan untuk menghitung jumlah seluruh node yang terdapat pada Binary Tree. Proses perhitungan dilakukan secara rekursif dengan mengunjungi setiap node pada child kiri dan child kanan hingga seluruh node berhasil dihitung.

Method print digunakan untuk menampilkan struktur Binary Tree dalam bentuk diagram sehingga hubungan antara root, child kiri, dan child kanan dapat dilihat dengan lebih jelas. Dengan adanya berbagai method tersebut, class ini dapat

digunakan untuk melakukan pengelolaan, pencarian, perhitungan jumlah node, serta traversal pada Binary Tree.

```
1 package pekan9_2511533005;
2
3 public class BTree_2511533005 {
4     private Node_2511533005 root_3005;
5     private Node_2511533005 currentNode;
6
7     public BTree_2511533005() {
8         root_3005 = null;
9     }
10
11     public boolean search(int data_3005) {
12         return search(root_3005, data_3005);
13     }
14
15     private boolean search(Node_2511533005 node_3005, int data_3005) {
16         if (node_3005.getData() == data_3005)
17             return true;
18         if (node_3005.getLeft() != null)
19             if (search(node_3005.getLeft(), data_3005))
20                 return true;
21         if (node_3005.getRight() != null)
22             if (search(node_3005.getRight(), data_3005))
23                 return true;
24         return false;
25     }
26
27     public void printInorder() {
28         root_3005.printInorder(root_3005);
29     }
30
31     public void printPreOrder() {
32         root_3005.printPreorder(root_3005);
33     }
34
35     public void printPostOrder() {
36         root_3005.printPostorder(root_3005);
37     }
38
39     public Node_2511533005 getRoot() {
40         return root_3005;
41     }
42 }
```

```
31     public void printPreOrder() {
32         root_3005.printPreorder(root_3005);
33     }
34
35     public void printPostOrder() {
36         root_3005.printPostorder(root_3005);
37     }
38
39     public Node_2511533005 getRoot() {
40         return root_3005;
41     }
42     public boolean isEmpty() {
43         return root_3005 == null;
44     }
45     public int countNodes() {
46         return countNodes(root_3005);
47     }
48     private int countNodes(Node_2511533005 node_3005) {
49         int count_3005 = 1;
50         if (node_3005 == null) {
51             return 0;
52         } else {
53             count_3005 += countNodes(node_3005.getLeft());
54             count_3005 += countNodes(node_3005.getRight());
55             return count_3005;
56         }
57     }
58     public void print() {
59         root_3005.print();
60     }
61     public Node_2511533005 getCurrent() {
62         return currentNode;
63     }
64     public void setCurrent(Node_2511533005 node_3005) {
65         this.currentNode = node_3005;
66     }
67     public void setRoot(Node_2511533005 root_3005) {
68         this.root_3005 = root_3005;
69     }
70 }
```

2.4 pengujian Operasi Binary Tree

Selanjutnya dibuat Class BTreeDriver yang berfungsi sebagai program utama untuk menguji implementasi Binary Tree yang telah dibuat sebelumnya. Pada method main, pertama-tama dibuat sebuah objek Binary Tree yang masih kosong. Kemudian jumlah simpul pada tree ditampilkan menggunakan method countNodes untuk menunjukkan bahwa belum terdapat node pada tree.

Setelah itu dibuat sebuah node dengan nilai 1 yang dijadikan sebagai root dari Binary Tree menggunakan method setRoot. Program kemudian kembali menampilkan jumlah simpul untuk menunjukkan bahwa tree telah memiliki satu node sebagai akar.

Selanjutnya dibuat beberapa objek node baru yang berisi nilai 2 sampai 9. Node-node tersebut kemudian dihubungkan satu sama lain menggunakan method setLeft dan setRight sehingga membentuk struktur Binary Tree. Hubungan antar node diatur secara manual dengan menentukan child kiri dan child kanan dari masing-masing node.

Setelah struktur tree selesai dibuat, method setCurrent digunakan untuk mengatur node aktif dengan mengambil nilai root dari tree. Data pada node aktif kemudian ditampilkan menggunakan method getCurrent dan getData.

Program juga menampilkan jumlah seluruh simpul yang terdapat pada Binary Tree menggunakan method countNodes. Setelah itu dilakukan traversal tree menggunakan tiga metode yang berbeda, yaitu InOrder, PreOrder, dan PostOrder. Hasil traversal ditampilkan ke layar sehingga urutan kunjungan node pada masing-masing metode dapat diamati dan dibandingkan.

Pada bagian akhir, method print dipanggil untuk menampilkan struktur Binary Tree dalam bentuk diagram pohon. Tampilan ini memudahkan pengguna dalam memahami hubungan antara root, child kiri, dan child kanan pada tree yang telah dibuat.

```

1 package pekan9_2511533005;
2
3 public class BTreeDriver_2511533005 {
4
5     public static void main(String[] args) {
6         //Membuat Pohon
7         BTree_2511533005 tree_3005 = new BTree_2511533005();
8         System.out.print("Jumlah Simpul awal pohon: ");
9         System.out.println(tree_3005.countNodes());
10        //menambahkan simpul data 1
11        Node_2511533005 root_3005 = new Node_2511533005(1);
12        //meniadikan simpul 1 sebagai root
13        tree_3005.setRoot(root_3005);
14        System.out.println("Jumlah simpul jika hanya ada root");
15        System.out.println(tree_3005.countNodes());
16        Node_2511533005 node2_3005 = new Node_2511533005(2);
17        Node_2511533005 node3_3005 = new Node_2511533005(3);
18        Node_2511533005 node4_3005 = new Node_2511533005(4);
19        Node_2511533005 node5_3005 = new Node_2511533005(5);
20        Node_2511533005 node6_3005 = new Node_2511533005(6);
21        Node_2511533005 node7_3005 = new Node_2511533005(7);
22        Node_2511533005 node8_3005 = new Node_2511533005(8);
23        Node_2511533005 node9_3005 = new Node_2511533005(9);
24        root_3005.setLeft(node2_3005);
25        node2_3005.setLeft(node4_3005);
26        node2_3005.setRight(node5_3005);
27        node4_3005.setRight(node8_3005);
28        root_3005.setRight(node3_3005);
29        node3_3005.setLeft(node6_3005);
30        node3_3005.setRight(node7_3005);
31        node6_3005.setLeft(node9_3005);
32        //Set root
33        tree_3005.setCurrent(tree_3005.getRoot());
34        System.out.println("menampilkan simpul terakhir: ");
35        System.out.println(tree_3005.getCurrent().getData());
36        System.out.println("Jumlah simpul: setelah simpul 7 ditambahkan");
37        System.out.println(tree_3005.countNodes());
38        System.out.println("InOrder: ");
39        tree_3005.printInorder();
40        System.out.println("\nPreorder: ");

```

```

32        //Set root
33        tree_3005.setCurrent(tree_3005.getRoot());
34        System.out.println("menampilkan simpul terakhir: ");
35        System.out.println(tree_3005.getCurrent().getData());
36        System.out.println("Jumlah simpul: setelah simpul 7 ditambahkan");
37        System.out.println(tree_3005.countNodes());
38        System.out.println("InOrder: ");
39        tree_3005.printInorder();
40        System.out.println("\nPreorder: ");
41        tree_3005.printPreOrder();
42        System.out.println("\nPostorder : ");
43        tree_3005.printPostOrder();
44        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
45        tree_3005.print();
46    }
47 }
48
49 }

```

BAB III

KESIMPULAN

3.1 KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data pohon (Binary Tree) dapat digunakan untuk menyimpan data secara hierarkis dengan hubungan parent dan child. Melalui implementasi Binary Tree dalam bahasa pemrograman Java, penulis dapat memahami cara membuat node, membangun struktur pohon, melakukan pencarian data, serta menghitung jumlah simpul yang terdapat pada pohon.

Selain itu, penulis juga mempelajari berbagai metode traversal pada Binary Tree, yaitu InOrder, PreOrder, dan PostOrder. Masing-masing metode memiliki urutan kunjungan node yang berbeda sehingga menghasilkan output yang berbeda pula. Pemahaman terhadap traversal ini sangat penting karena menjadi dasar dalam proses pengolahan dan pencarian data pada struktur pohon.

Pada praktikum ini juga dipelajari algoritma Breadth First Search (BFS) dan Depth First Search (DFS) yang digunakan untuk melakukan penelusuran pada graf. BFS melakukan penelusuran secara melebar menggunakan queue, sedangkan DFS melakukan penelusuran secara mendalam menggunakan rekursi atau stack. Dari hasil pengujian, kedua algoritma mampu mengunjungi seluruh node pada graf, namun menghasilkan urutan penelusuran yang berbeda sesuai dengan karakteristik masing-masing algoritma.

Dengan dilaksanakannya praktikum ini, penulis memperoleh pemahaman yang lebih baik mengenai konsep struktur data pohon, graf, traversal, serta implementasi algoritma BFS dan DFS dalam pemrograman Java.

DAFTAR PUSTAKA

- [1] Oracle, "Queue (Java Platform SE 17)," Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html> (diakses: 10 Juni 2026).
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] Wikipedia, "Breadth first search," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Breadth-first_search (diakses: 10 Juni 2026).
- [4] Wikipedia, "Depth first search," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Depth-first_search (diakses: 10 Juni 2026).